BERT++

Leveraging FSDP and Modern Techniques for Efficient Training

Drew Patrick

Kennesaw State University

College of Computing Science and Engineering

Apatri12@students.kennesaw.edu

KSU ID: Apatri12

Abstract

BERT++ is an enhanced training pipeline for BERT-Large models that integrates advanced system-level tools and optimization techniques to overcome hardware limitations. We implemented a 24-layer, 16-head Transformer (matching the BERT-large architecture) with ~340 million parameters [1] and are currently in the beginning stages of training it on modest hardware by leveraging Fully Sharded Data Parallel (FSDP) and gradient checkpointing for memory optimization. Our approach shards model parameters across GPUs to drastically reduce per-device memory use, while recomputing activations on the fly to save memory with no impact on model accuracy. We also employed Automatic Mixed Precision (AMP) and FlashAttention to accelerate training, which will yield significant speed-ups (FlashAttention provides ~15% faster training on BERT-large sequences) and reduced memory bandwidth overhead. Furthermore, we integrated system-level tooling (SSH access, and automated Weights & Biases logging) to streamline experiment management. We speculate that this model will demonstrate that careful engineering can enable the training of large-scale Transformer models on limited hardware resources.

Introduction

Recent advances in NLP have been driven by large Transformer-based language models. BERT (Bidirectional Encoder Representations from Transformers) introduced by Devlin et al. [1] revolutionized NLP by applying bidirectional Transformer training to language modeling, achieving state-of-the-art performance on a range of tasks. The BERT-Large variant (24 layers, 16 attention heads, ~340M parameters) significantly improves accuracy on downstream tasks by capturing deep contextual relationships in text. However, this improved performance comes at the cost of substantial computational and memory demands – deploying or training BERT-Large is challenging on resource-constrained hardware. Over 24 Transformer layers, the model requires enormous GPU memory for storing parameters and activations, and training such a model from scratch can take days even on accelerator-rich setups [3].

BERT++ is our project's solution to this problem: an enhanced training pipeline designed to make BERT-Large training feasible and efficient on modest hardware setups. Instead of downsizing the model, we focus on scaling the training process using a combination of software optimizations and tooling. In particular, we integrate PyTorch's Fully Sharded Data Parallel (FSDP) algorithm to distribute the model across multiple GPUs, dramatically reducing memory overhead per device. We pair this with gradient checkpointing (also known as activation checkpointing) to trade extra computation for lower memory usage during backpropagation [11]. Additionally, we incorporate Automatic Mixed Precision (AMP) to use lower-precision arithmetic for faster computation and lower memory footprint [7], and we implement FlashAttention – a recent attention algorithm that minimizes memory reads/writes – to speed up the Transformer's attention mechanism [10]. These techniques collectively address the bottlenecks of training large models, enabling BERT++ to train a 340M-parameter Transformer on hardware previously considered insufficient.

Beyond model optimizations, BERT++ also emphasizes robust engineering practices. We set up a remote training environment and experiment tracking: using SSH for stable remote session control, and leveraging Weights & Biases (W&B) for experiment tracking and artifact logging. W&B is a platform and library for machine learning engineers to run experiments and log artifacts. In our scripts we automated the login and ensured that metrics, hyperparameters, and model snapshots are logged in real-time to an online dashboard.

In summary, this report details how we combined state-of-the-art distributed training and memory optimization techniques with practical tooling to show how to successfully train a BERT-large model variant under tight hardware constraints. We first review related work on large-model training and prior BERT improvements. We then describe the BERT++ methodology, including the model architecture and the training pipeline enhancements. We discuss the advantages of our approach – such as improved memory efficiency and scalability –

as well as its limitations (added complexity and overhead). We conclude with insights gained and potential future improvements. Our paper proposes that with careful planning and modern tools, even large Transformers can be trained on modest hardware, democratizing advanced NLP capabilities.

Related Works

BERT:

The original BERT model [1] set new benchmarks by using a multi-layer bidirectional Transformer encoder to learn deep context representations. BERT's two sizes, Base (110M parameters) and Large (340M), demonstrated the benefit of scaling model size for NLP tasks. Following BERT, researchers explored both larger pre-training and model efficiency. RoBERTa (Liu et al., 2019) showed that with longer training on more data and removal of certain tasks, the BERT-large architecture could yield even better results, highlighting the importance of training procedure. On the other hand, compression approaches like DistilBERT and ALBERT reduced BERT's size or parameter redundancy to make inference and fine-tuning more efficient. However, they did sacrifice some capacity. These works illustrate two paths: scaling up for performance or trimming down for efficiency. BERT++ aligns with the former in spirit (retaining a large model for performance) but tackles the efficiency problem through improved training techniques rather than reducing model size.

Large-Scale Training Techniques:

Training very large models has pushed the development of new distributed training algorithms. Traditional data-parallel training duplicates model parameters on each GPU, which becomes infeasible for models with hundreds of millions or billions of parameters due to memory limits. To address this, ZeRO (Zero Redundancy Optimizer) and related strategies shard optimizer states and gradients across workers, reducing memory duplication. Fully Sharded Data Parallel (FSDP) extends this idea by sharding all model parameters (and gradients and optimizer states) across GPUs, so that each GPU holds only a fraction of the model at any given time. Originally implemented in the FairScale library by Facebook, FSDP enables training models that are orders of magnitude larger using fewer GPUs. Our work leverages PyTorch's built-in FSDP implementation, building on these advances. Additionally, gradient checkpointing is a generic technique to save memory: instead of storing every layer's activations for backpropagation, it stores only a subset and recomputes the rest during the backward pass [11]. This approach yields mathematically equivalent training with no accuracy loss, at the cost of extra computation.

Efficient Attention Mechanisms:

Transformers face a quadratic cost in sequence length for self-attention, which can become a bottleneck. Many approximate attention methods have been proposed to mitigate this for long sequences, but they often trade accuracy for speed. In contrast, FlashAttention is an "exact" attention algorithm [2] that is optimized for modern hardware's memory hierarchies [10]. By tiling operations and reducing memory reads/writes between GPU high-bandwidth memory and on-chip SRAM, FlashAttention achieves the same results as standard attention while greatly improving speed and memory usage. Dao et al. reports that FlashAttention provides a 15% end-to-end speedup in BERT-large training (512 token sequences) compared to the previous state-of-the-art, with even larger gains for longer sequences [10]. This makes it very attractive for accelerating large-model training. In our project, we integrate FlashAttention to replace the default attention mechanism, taking advantage of these efficiency gains.

Experiment Tracking and Reproducibility:

Modern machine learning projects often use tools like Weights & Biases (W&B) or TensorBoard to log training progress and results. Weights & Biases in particular has emerged as a robust experiment tracking platform that logs metrics, hyperparameters, and model artifacts in real-time. Using such tools is now considered the best practice for ensuring reproducibility and for analyzing model training behaviors (loss curves, validation performance) post-hoc. We adopt W&B in BERT++ to record all aspects of training, which situates our work in line with common machine learning practices.

In summary, BERT++ draws inspiration from prior research on scaling Transformers (FSDP, ZeRO, etc.), memory-saving techniques (activation checkpointing), and efficient computation (mixed precision, FlashAttention), combining them with practical experiment management tools. Our contribution is in the "integration and validation" of these techniques in a single framework aimed at training a large model under constrained conditions.

Methodology

Our model is a transformer encoder only type of model. The initial embedding layer will convert each token ID into a vector. We are utilizing the parallel block, which essentially puts the attention mechanism on a parallel plane with the feed forward networks. Because of this architecture, vectors from the embedding layer are sent directly to the attention blocks and the feed-forward blocks. The multi-head attention blocks first compute the queries and key vectors.

XPOS, which is a rotational transformation that encodes token position in a sequence, is then applied [8]. These new rotational vectors then undergo the dot-product attention operation across all heads utilizing the FlashAttention mechanism mentioned earlier. The dot-product is then concatenated with all the dot products computed across all the attention heads. In parallel, the input is also sent to the feed-forward blocks, which will be utilizing SwiGLU activation functions [6]. The outputs from both the multi-head attention block and the feed-forward blocks are then combined and fused with the layers original input via residual connections before being fed into the next layer. After 24 layers, the output is passed into a normalizing layer. This rescales and re-centers the data for model stability purposes. The output from the normalizing layer is passed to the output layer that contains SoftMax activation functions which yields a probability distribution over the entire vocabulary. The loss is then calculated (cross-entropy) for the masked positions (15% of the tokens will be masked) and this loss is then used to update weights across the entire model via the AdamOptimizer. Each attention block has 16 heads, while the FFN contains 4 layers with 4096 neurons in each layer. These are the initial hyper parameters for our model. We expect that we will be able to increase these hyperparameters without straining our hardware due to the efficiency of our model. We do not expect that an increase in model complexity will result in overfitting due to the massive size of the dataset we are utilizing. This dataset is streamed instead of downloaded via Hugging Face, which hands us chunks from C4 [4] and The Pile [5] on demand—fifty-fifty, clean, and shuffled. BertTokenizerFast does on-the-fly subword splits, respecting the [MASK] token, trimming anything past 512 tokens and padding the rest so every batch shares a shape the model likes.

Advantages

FSDP shards weights across GPUs while gradient checkpointing recomputes activations on-the-fly, letting a 340 M-parameter, 24-layer model slip under the memory ceiling of three mid-range cards without touching accuracy. Mixed precision and FlashAttention then turn those saved bytes into real throughput, trimming step time (≈1.5 × faster with AMP, another ~15 % from FlashAttention) so training becomes days and not weeks.

Disadvantages

That efficiency buys complexity. FSDP's sharding choreography, checkpoint gathering, and multi-rank sync add more moving parts than a vanilla single-GPU script. Checkpointing also injects 20-ish % extra compute per batch, while FSDP's all-to-all traffic can nibble at utilization if interconnects lag. Finally, "modest hardware" still means at least three modern GPUs—one eight-gig card won't cut it—and the rapid churn of PyTorch and FlashAttention versions keeps the maintenance treadmill running.

Conclusion

BERT++ attempts to prove that training a full-scale BERT-Large doesn't require a server farm, just smart engineering. By pairing FSDP sharding with gradient checkpointing, we'll attempt to squeeze 340 M parameters into 3 GPUs; AMP and FlashAttention will then claw back the speed those tricks would have cost. Checkpoints and W&B logging turn crashes into hiccups and experiments into shareable, reproducible artifacts. The takeaway is simple: memory limits are hurdles, not walls, when you're willing to trade a bit of extra computation and tooling complexity.

Looking ahead, task-specific fine-tuning (GLUE, SQuAD, etc.) will let us benchmark exactly how much headroom this pipeline buys over baseline BERT. We also plan to test deeper off-loading strategies (ZeRO-Infinity, asynchronous saves) and keep pace with PyTorch's evolving FSDP and built-in flash attention. But even as-is, BERT++ stands as a template for "big model on modest gear," lowering the entry fee for serious NLP research and nudging the field a step closer to true hardware democracy.

References

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint* arXiv:1810.04805, Oct. 2018.

[2] A. Vaswani *et al*., “Attention is all you need,” in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.* (NeurIPS), Long Beach, CA, USA, Dec. 2017, pp. 5998–6008.

[3] D. Narayanan *et al*., “Scaling language-model training to a trillion parameters using Megatron,” NVIDIA Tech. Rep., Sep. 2021. [Online]. Available: https://developer.nvidia.com/blog/scaling-language-model-training-to-a-trillion-parameters-using-megatron

[4] C. Raffel *et al*., “Exploring the limits of transfer learning with a unified text-to-text transformer,” *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2020.

[5] L. Gao *et al*., “The Pile: An 800 GB dataset of diverse text for language modeling,” *arXiv preprint* arXiv:2101.00027, Jan. 2021.

[6] N. Shazeer, “GLU variants improve transformer,” *arXiv preprint* arXiv:2002.05202, Feb. 2020.

[7] P. Micikevicius *et al*., “Mixed-precision training,” in *Proc. 6th Int. Conf. Learn. Represent.* (ICLR), Vancouver, BC, Canada, Apr. 2018.

[8] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, “RoFormer: Enhanced transformer with rotary position embedding,” *Neurocomputing*, vol. 568, pp. 127063-1–127063-10, Feb. 2024.

[9] D. Vilar *et al*., “Prompting PaLM for translation: Assessing strategies and performance,” *arXiv preprint* arXiv:2211.09102, Nov. 2023.

[10] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, C. Ré, and K. Keutzer, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” *arXiv preprint* arXiv:2205.14135, May 2022.

[11] T. Chen, B. Xu, C. Zhang, and C. Guestrin, “Training deep nets with sublinear memory cost,” *arXiv preprint* arXiv:1604.06174, Apr. 2016.